

13강. PV-PVC



이영곤



기본 개념

- ✓ PV (PersistentVolume)
 - 👉 실제 스토리지 (디스크 자원)
- ✓ PVC (PersistentVolumeClaim)
 - 👉 사용자가 요청하는 스토리지 (요구사항)

문제 정의

- ✓ Pod → 디스크 직접 연결시
 - 👉 Pod 삭제하면 데이터도 같이 삭제됨
 - 👉 스토리지 교체 어려움
 - 👉 클라우드/온프레미스 종속

해결책

- ✓ Pod → PVC → PV → 실제 스토리지
 - 👉 Pod와 스토리지 분리
 - 👉 데이터 영속성 확보
 - 👉 인프라 독립성 확보

정적 방식

- ① 관리자: PV 생성
- ② 사용자: PVC 생성
- ③ Kubernetes: 조건 맞으면 바인딩
- ④ Pod: PVC 사용

동적 방식 (StorageClass 포함)

- ✓ PVC 생성
 - StorageClass
 - PV 자동 생성
 - 바인딩 (조건이 맞아야 함)
 - Pod 사용

동적 방식 사례

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast
resources:
  requests:
    storage: 5Gi
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: rancher.io/local-path
```

- StorageClass: 어떤 종류의 저장소를, 어떤 방식으로 자동 생성할 것인지 정의한 템플릿

PVC

resources:

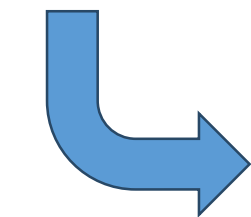
requests:

storage: 1Gi

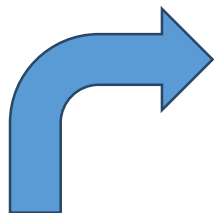
PV

capacity:

storage: 2Gi



조건이 맞음



PVC (1Gi 요청)
→ PV (2Gi 있음)
→ Bound

바인딩

- ✓ 바인딩 (Binding)
- ✓ PVC ↔ PV 연결
- ✓ 바인딩 조건:
 - 용량
 - accessModes: 이 저장소를 몇 개의 노드/Pod가 동시에 읽고 쓸 수 있는가?
 - storageClass

액세스 모드	의미
ReadWriteOnce	하나의 노드에서 읽기/쓰기 가능
ReadOnlyMany	여러 노드에서 읽기 가능
ReadWriteMany	여러 노드 읽기/쓰기 가능

PersistentVolumeReclaimPolicy

- ✓ PVC가 삭제된 이후 PV(디스크)를 어떻게 처리할지 결정하는 정책
- ✓ PVC 삭제 시 작동
- ✓ PVC 삭제 → PV 상태 Released → Reclaim Policy 적용

정책	의미
Retain	디스크 유지
Delete	디스크 폐기
Recycle	포맷 후 재사용

PV 생성

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: demo-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /tmp/demo-pv-data
```

PVC 생성

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
resources:
  requests:
    storage: 500Mi
```

PV-PVC 바인딩: 매칭 조건에 따른 자동 바인딩

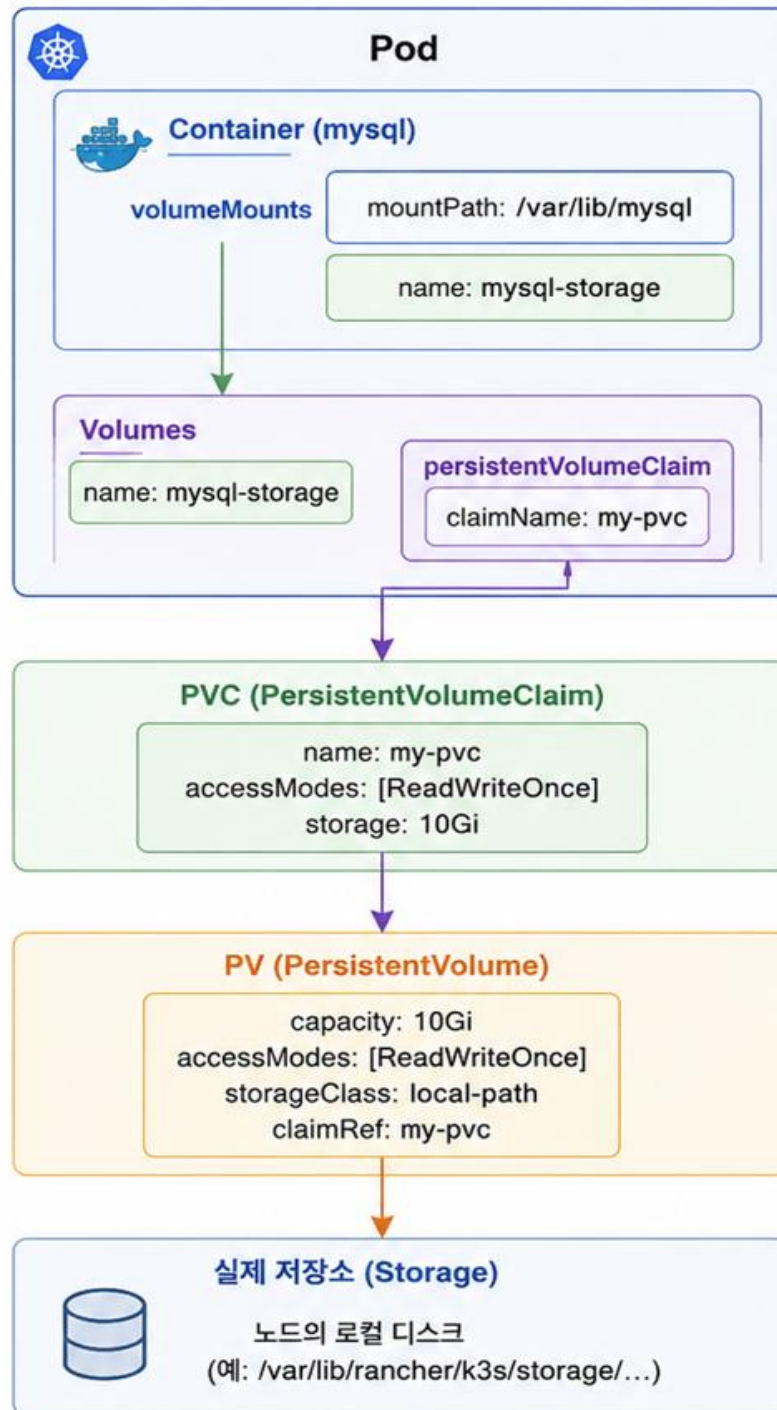
- ✓ storageClassName
 - PVC: manual
 - PV: manual → 일치
- ✓ 용량 (capacity $\geq$ request)
 - PVC: 500Mi 요청
 - PV: 1Gi 제공 → 충분함
- ✓ accessModes
 - PVC: ReadWriteOnce
 - PV: ReadWriteOnce → 일치

PVC를 이용해 DB 생성

```
spec:
  containers:
    - name: mysql
      image: mysql:8.0
      ports:
        - containerPort: 3306
      env:
        - name: MYSQL_ROOT_PASSWORD
          value: root1234
        - name: MYSQL_DATABASE
          value: bookdb
      volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
  volumes:
    - name: mysql-storage
      persistentVolumeClaim:
        claimName: my-pvc
```

volume: Pod 내부에서
사용하는 가상 볼륨이름

- 1 Container가 볼륨을 마운트
- 2 Volume이 PVC를 참조
- 3 PVC가 PV를 바인딩
- 4 PV가 실제 저장소를 사용



Pod YAML (일부)

```
spec:
  containers:
    - name: mysql
      image: mysql:8.0
      volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql
  volumes:
    - name: mysql-storage
      persistentVolumeClaim:
        claimName: my-pvc
```

PVC YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: local-path
```

PV (자동 생성 예)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pvc-xxxxxx
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
  claimRef:
    name: my-pvc
```

필요성

- ✓ PV 동적 프로비저닝을 위해
- ✓ StorageClass 없을 때
 - 관리자: PV 미리 생성
 - 사용자: PVC 생성 → 매칭
 - 문제: PV를 미리 만들어야 함
 - 자동화 불가
 - 운영 부담 큼
- ✓ StorageClass 있을 때
 - StorageClass로부터 PV 자동 생성

개념	역할
PV	실제 디스크
PVC	사용자 요청
StorageClass	디스크 생성 방법 정의

사용자는 PVC만
정의해서 사용

핵심 기능

- ✓ 어떤 스토리지를 쓸지 결정
 - AWS → EBS
 - GCP → Persistent Disk
 - Docker Desktop → local-path
- ✓ 어떻게 만들지 정의
 - SSD / HDD
 - 크기
 - IOPS
- ✓ replication 여부
- ✓ 생성 트리거
 - PVC → StorageClass → Provisioner 실행 → PV 생성

예제

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
```

StorageClass

```
spec:
  storageClassName: fast-storage
resources:
  requests:
    storage: 1Gi
```

PVC

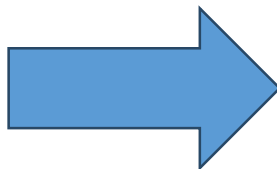
Default storageclass를 이용해 PV 자동 생성

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: sc-demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

- `kubectl get storageclass` ⇒ default storageclass 확인
- `kubectl get pvc`
- `kubectl get pv`

POD에서 PVC 선언

```
volumeMounts:  
  - mountPath: /data  
    name: storage  
volumes:  
  - name: storage  
    persistentVolumeClaim:  
      claimName: sc-demo-pvc
```



PV가 자동 생성되어
POD에서 사용 가능

- `kubectl exec -it mysql-pod -- bash` ⇒ pod 내로 들어 가서
- `df -h` ⇒ 스토리지 할당 내역 확인
- `/data`는 컨테이너 내부 경로지만, 실제로는 PV에 연결된 외부 스토리지

개념

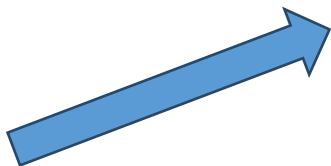
- ✓ 상태(State)를 가지는 애플리케이션을 안정적으로 실행하기 위한 Kubernetes 리소스
- ✓ StatefulSet = Pod의 “정체성(Identity) + 저장소 + 순서”를 보장하는 컨트롤러

필요성

- ✓ 일반 Pod / Deployment
 - Pod 삭제 → 새 Pod 생성 (이름 변경, IP 변경, 디스크 변경)
 - 👉 DB 데이터 유실 가능
 - 네트워크 주소 변경
 - 순서 보장 없음

StatefulSet

✓ 이름 고정



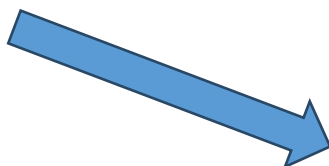
mysql-0 → 항상 mysql-0
mysql-1 → 항상 mysql-1

✓ 스토리지 고정



mysql-0 → pvc-mysql-0
mysql-1 → pvc-mysql-1

✓ 순서 보장



생성: 0 → 1 → 2
삭제: 2 → 1 → 0

StatefulSet vs Deployment

항목	Deployment	StatefulSet
Pod 이름	랜덤	고정
스토리지	공유 or 불명확	Pod별 고정
순서	없음	있음
대상	stateless	stateful

필요한 경우

- ✓ MySQL / PostgreSQL, Kafka, Redis (cluster), Elasticsearch

StatefulSet 예제

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  replicas: 3

  volumeClaimTemplates:
  - metadata:
      name: mysql-data
    spec:
      accessModes:
      - ReadWriteOnce

      storageClassName: fast-storage

    resources:
      requests:
        storage: 10Gi
```

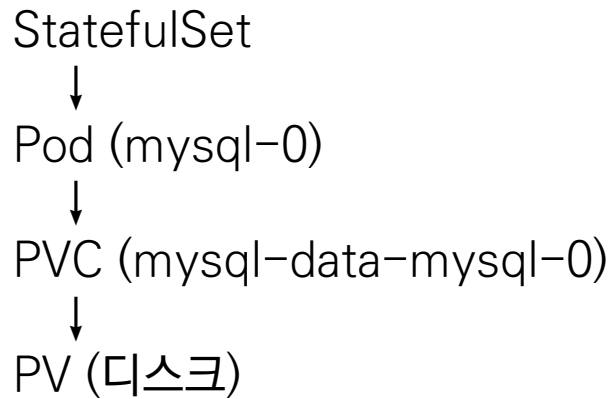
```
graph TD
    A[mysql-0 생성] --> B[mysql-data-mysql-0 PVC 생성]
    B --> C[PV 자동 생성]
    C --> D[mysql-1 생성]
    D --> E[mysql-data-mysql-1 PVC 생성]
    E --> F[PV 자동 생성]
    F --> G[mysql-2 생성]
    G --> H[mysql-data-mysql-2 PVC 생성]
    H --> I[PV 자동 생성]
```

mysql-0 생성
↓
mysql-data-mysql-0 PVC 생성
↓
PV 자동 생성

mysql-1 생성
↓
mysql-data-mysql-1 PVC 생성
↓
PV 자동 생성

mysql-2 생성
↓
mysql-data-mysql-2 PVC 생성
↓
PV 자동 생성

StatefulSet 작동 순서



📌 Pod와 디스크가 1:1 고정

StatefulSet 과 DB

✓ 왜 DB에 필수인가?

- DB는 특정 디스크에 데이터 저장
- 노드별 역할 존재 (Primary / Replica)

👉 Pod가 바뀌면 안됨

Headless service 개념

Headless Service는 로드밸런싱 없이 Pod 각각의 주소를 그대로 노출하는 Service

✓ 일반 Service

- Client → Service → Pod (랜덤 분산)

👉 특징:

하나의 IP (ClusterIP)

로드밸런싱 수행

Pod 개별 구분 불가

✓ Headless Service

- Client → Pod 직접 접근

👉 특징:

ClusterIP 없음 (clusterIP: None)

DNS가 Pod 목록을 그대로 반환

로드밸런싱 없음

Headless service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  clusterIP: None # headless 서비스 정의
  selector:
    app: mysql
  ports:
    - port: 3306
```

Headless service 를 statefulset 과 같이 사용해야 하는 이유

✓ 일반 Service 사용 시,

- mysql-service → mysql-0 or mysql-1 랜덤 연결

👉 문제:

어떤 Pod인지 알 수 없음

Primary/Replica 구분 불가

DB 클러스터 구성 불가능

✓ Headless Service 사용 시,

- mysql-0.mysql, mysql-1.mysql

👉 각각 직접 접근 가능

DNS가 파악



⟨pod-name⟩.⟨service-name⟩

Headless service 를 사용해야 하는 이유

- ✓ Headless Service 아니면,
 - Pod 개별 접근 불가
 - Pod별 DNS 이름 없음
 - 클러스터 구성 실패: DB, Kafka, ZooKeeper 등 Pod명 정확히 알아야 함
- ✓ Headless Service 사용 시,
 - Pod별 DNS 제공
 - 클러스터 구성 가능
 - 안정적인 연결

사례

Headless Service (mysql)



StatefulSet (replicas=2)

└─ mysql-0 (Primary) → PVC: mysql-data-mysql-0
 └─ mysql-1 (Replica) → PVC: mysql-data-mysql-1

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: mysql

PVC 뒷부분 이름+ 생성
순번(0,1,2..) ⇒ Pod명

spec:

replicas: 2

volumeClaimTemplates:

- metadata:

name: mysql-data

PVC 앞부분 이름

PVC 명

사례

- ✓ Headless Service
 - Pod별 DNS 제공
 - mysql-0.mysql, mysql-1.mysql 접근 가능
- ✓ StatefulSet
 - Pod identity 유지
 - 순서 보장
 - PVC 자동 생성
- ✓ volumeClaimTemplates
 - Pod마다 독립 스토리지 생성

StatefulSet (MySQL)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 2
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
```

```
env:
  - name: MYSQL_ROOT_PASSWORD
    value: root1234
  - name: MYSQL_DATABASE
    value: testdb
```

```
volumeMounts:
  - name: mysql-data
    mountPath: /var/lib/mysql
```

```
volumeClaimTemplates:
  - metadata:
      name: mysql-data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 1Gi
```

Headless Service로 MySQL StatefulSet에 직접 접근하는 사례

✓ 준비 사항:

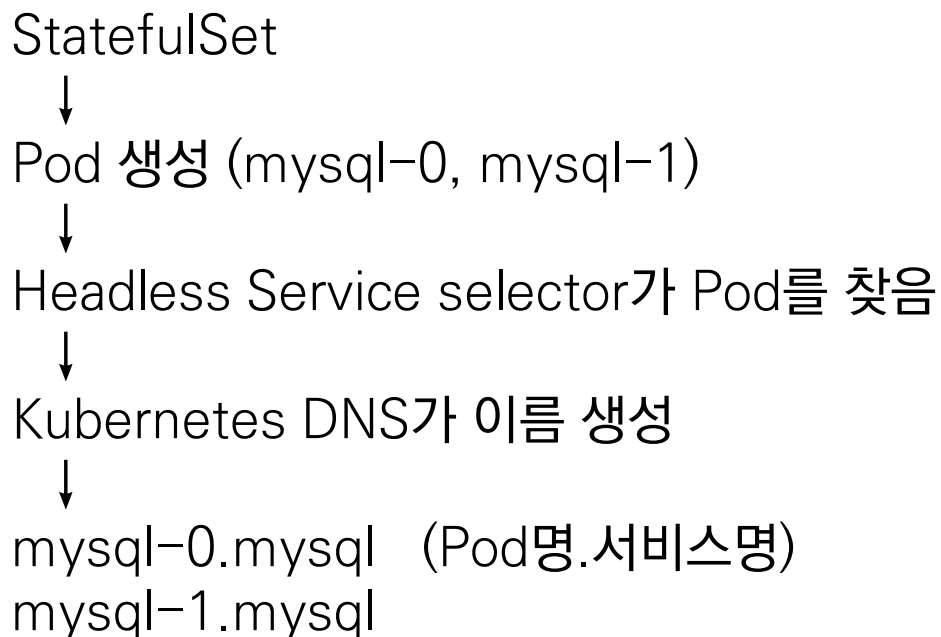
- Headless Service (mysql)
- StatefulSet (mysql, replicas ≥ 1)
- `kubectl get svc mysql`
- `kubectl get pods`

✓ Headless Service가 있으면 자동 생성됨:

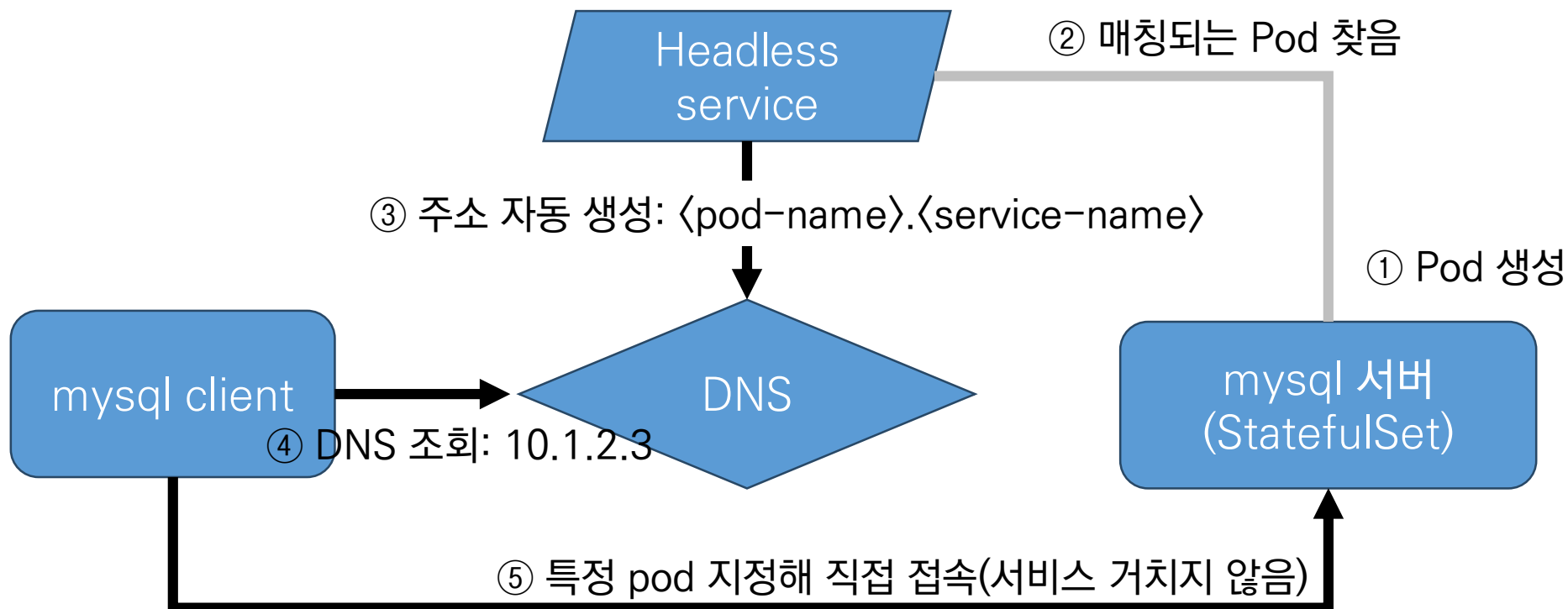
- `mysql-0.mysql`
- `mysql-1.mysql`
- `<pod-name>.<service-name>`

Headless Service로 MySQL StatefulSet에 직접 접근하는 사례

✓ 프로세스



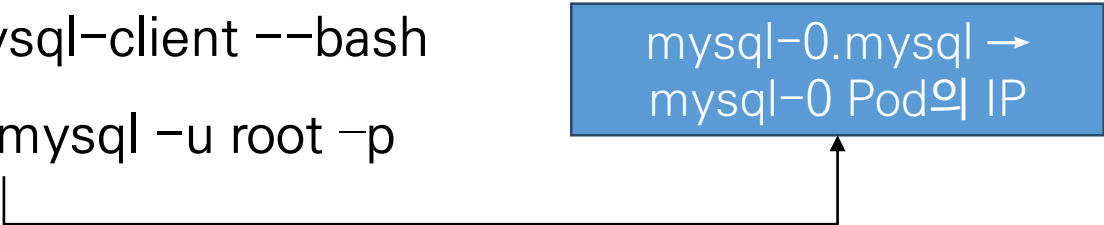
Headless Service로 MySQL StatefulSet에 직접 접근하는 사례



MySQL 클라이언트 생성 후 이를 통해 MySQL(StatefulSet) 접속

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql-client
spec:
  containers:
    - name: mysql-client
      image: mysql:8.0
      command: ["sleep", "3600"]
```

- ✓ `kubectl apply -f client.yaml`
- ✓ `kubectl exec -it mysql-client --bash`
- ✓ `mysql -h mysql-0.mysql -u root -p`



mysql-0.mysql →
mysql-0 Pod의 IP

가장 단순한 방법 (직접 접속)

```
spring.datasource.url=jdbc:mysql://외부DB주소:3306/testdb  
spring.datasource.username=xxx  
spring.datasource.password=xxx
```

- ✓ 코드에 DB 주소 넣음
- ✓ 환경(dev/prod) 바뀌면 수정 필요
- ✓ 쿠버네티스 스타일이 아님

ExternalName Service (DNS 방식)

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-external
spec:
  type: ExternalName
  externalName: mydb.xxxxxx.rds.amazonaws.com
```

```
spring.datasource.url=jdbc:mysql://mysql-external:3306/testdb
```

- ✓ DNS CNAME 역할
- ✓ 가장 간단
- ✓ TCP 그대로 전달

ClusterIP + Endpoints (IP 직접 매핑)개념 -> 외부 IP를 내부 서비스 연결

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-external
spec:
  ports:
    - port: 3306
```

```
apiVersion: v1
kind: Endpoints
metadata:
  name: mysql-external
subsets:
  - addresses:
      - ip: 192.168.0.100
    ports:
      - port: 3306
```

Service 이름과 Endpoints
이름이 같아야 연결

- Service가 Endpoints를 참조해서 트래픽을 전달
- 사용: mysql-external:3306

Secret + ConfigMap

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
stringData:
  username: user
  password: pass
```

```
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

감사합니다.